

Auditoria Técnica

Projeto P2P (Procure-to-Pay)

HRG3 / Guess

Diagnóstico completo pré-produção

Emitido em 01/06/2026

Análise crítica de arquitetura, segurança, qualidade de código, integrações ERP, performance, UX, testes, build e deploy. 115 achados classificados por severidade (CRÍTICA / ALTA / MÉDIA / BAIXA).

Sumário

1. Resumo executivo da saúde do projeto
2. Lista priorizada de problemas
3. Classificação por severidade — totais
4. Plano de correção por fases
5. Checklist do que precisa ser corrigido antes de produção
6. Recomendações de arquitetura
7. Recomendações de segurança
8. Recomendações de UX/UI
9. Recomendações de testes
10. Perguntas que precisam ser respondidas antes de mexer no código

Diagnóstico baseado em varredura profunda do backend (NestJS), frontend (React/Vite) e infra/integrações por três agentes especializados em paralelo. Nada foi modificado.

1. Resumo executivo da saúde do projeto

Veredito honesto: produto sólido em fundamento, mas operacionalmente NÃO está pronto para produção real.

Boas notícias primeiro: a arquitetura macro está coerente. Backend NestJS com módulos bem segmentados por domínio (auth, requisitions, purchase-orders, fiscal-documents, integration, financial), Prisma com schema cuidadoso (39 models, 42 índices, soft delete consistente em entidades-chave, snapshots de nomes/códigos do ERP — boa decisão), ApiExceptionHandler traduzindo erros do Linx/Prisma para PT-BR, AuditInterceptor com mascaramento de PII (parcial), AccountLockoutService com janelas crescentes, SecretService AES-256-GCM previsto, cookie httpOnly em PROD, ThrottlerModule global + por-endpoint sensível, Helmet, Turnstile no login. Frontend cuidado: comentários em PT-BR explicando o "porquê", ErrorBoundary global, RequireAuth/RequireProfile composáveis, React Query bem configurado, filtros persistidos em URL (na tela de Pedidos do Linx), nav.ts declarativo com badges. Não há dangerouslySetInnerHTML em nenhum lugar do frontend (verificado).

Más notícias: a borda do sistema está em "máquina do dev". Resumindo os 4 grandes problemas:

(A) Segredos & autenticação em estado vulnerável. JWT_SECRET é literalmente dev_trocar_por_segredo_forte_em_producao_64_chars em PROD e HML (mesmo valor). SECRET_ENCRYPTION_KEY está ausente — então o SecretService que deveria criptografar senhas SMTP em AES-256-GCM está em modo passthrough silencioso, gravando senhas SMTP de empresas em texto puro na coluna company_erp_configs.smtpPassword. Senhas reais do SQL Server PROD, credenciais LDAP do AD ("P2P Service") e API key/secret da Qive vivem em .env num share de rede (I:\p2p). Em paralelo, o modo demo do frontend é ativável a partir da tela de login em produção (sem gating por env), e o adapter axios intercepta todas as chamadas devolvendo dados mockados — qualquer um vira "ADMIN" simulado. E o refresh token é gravado em localStorage mesmo quando o modo cookie httpOnly está ativo — anula 80% do benefício da migração.

(B) Integração ERP transacionalmente frágil. LinxErpService.gravarPedidoCompra admite explicitamente que não usa \$transaction por causa de conflito com triggers do Linx. Se cair entre o INSERT em COMPRAS e o INSERT em COMPRAS_CONSUMIVEL, fica pedido fantasma no ERP — recovery por OBS LIKE só evita duplicar cabeçalho, não corrige itens faltando. Em ProductOrdersPaService.approve/reject/reschedule, há um bug silencioso pior: \$transaction(async () => { this.prisma.\$executeRawUnsafe(...) }) ignora o tx recebido e usa this.prisma direto — a transação é falsa, falha parcial gera estado inconsistente sem detecção. Cross-DB com \$queryRawUnsafe interpola \${erpDbName} em 5 services sem usar a função safeDbName() que existe em financial.service.ts — defense-in-depth quebrada.

(C) Deploy & operações sem rede de segurança. Backend sobe via Start-Process node no Windows (RUNBOOK), sem PM2 (apesar do pm2.config.js existir), sem auto-restart, sem rotação de log, sem rollback. Logs sem logrotate crescem indefinidamente. Sem Sentry/APM — bugs em produção se descobrem por ticket. Cron jobs em memória (syncStates: Map) sem lock distribuído — perdem progresso no restart e duplicam se subir 2 instâncias. Sem backup documentado do P2P_DB. Pool Prisma com defaults (max:10) — satura em pico. NODE_ENV=development no .env que é PROD. Swagger possivelmente exposto em PROD. Migrations PROD via prisma migrate deploy mas HML via apply-hml-migrations.js paralelo (drift latente).

(D) Qualidade & dívida acumulada. 5 Pages monolíticas de 600–1055 LOC (RequisitionFormPage = 1055 LOC com 11 useState e lógica de save tripla). Cobertura de teste praticamente zero: 5 specs no

backend (1 stub do scaffold), 0 testes unitários no frontend, 1 smoke e2e Playwright. SupplierCombobox duplicado divergente em `pages/financeiro/` e `pages/requisitions/`. `lib/demo/handlers` com 256+ ocorrências de `any` (aceitável em mocks, mas zero tipagem). `bullmq+@nestjs/bullmq` instalados mas zero uso. `ReactQueryDevtools` carregado em build de produção. Bundle único de 791KB sem code-splitting (`React.lazy` ausente em todo o projeto). Diversos scripts ad-hoc (`inspect-*.ts`, `seed-*.js`) versionados misturados com código de produção.

Em UX, vazam jargões para o usuário (LANCAMENTO, ITP, IAD, PA, DDA, Microvix, `accessKey`, códigos crus de status como `M/CP/DP`). 5 `confirm()` nativos do browser em ações destrutivas e até em aprovação de Pedido PA. Mensagens de erro genéricas ("Erro inesperado. Tente novamente.") em fallback. Há um bug de rota óbvio: `FiscalDocumentDetailPage.tsx:221` faz `navigate` para `/pedidos-compra/...` mas a rota real é `/pedidos/:id` — clicar no link de PC vinculado leva ao dashboard. Acessibilidade pobre (22 ocorrências total de `aria-*/role=`).

Avaliação por área (0–10)

Área	Nota	Observação
Arquitetura macro	7	Bem segmentado, dois pontos fracos (sem service-layer ERP, state em memória)
Qualidade backend	6	Boa intenção, muitos any/queryRawUnsafe sem proteção
Qualidade frontend	6	Pages gigantes, hooks bem nomeados, duplicações pontuais
Modelagem de dados	8	Schema cuidadoso, snapshots e índices certos
Segurança autenticação	4	JWT_SECRET default, refresh em localStorage, demo bypass
Segurança de dados	3	SECRET_ENCRYPTION_KEY ausente → SMTP em plain text
Integração ERP	5	Funcional mas sem transação real, recovery frágil
Tratamento de erros	6	Filter global bom, msgs genéricas e leak de detalhes técnicos
UX/UI	6	Boa em fluxos novos, vazamento de jargão em telas financeiras
Acessibilidade	3	Quase nada
Testes	2	Praticamente inexistentes
Build & Deploy	3	Sem PM2, sem rollback, sem rotação
Observabilidade	2	Sem error tracking, sem APM, sem métricas
Escalabilidade	3	Crons em memória, pool DB pequeno, sem lock
Manutenibilidade	6	Comentários em PT-BR ajudam, Pages enormes atrapalham

Saúde geral: 5/10. Funciona como MVP. NÃO está pronto para subir em produção real com dados sensíveis do Linux sem mitigar o bloco P0 (12 itens) abaixo.

2. Lista priorizada de problemas

CRÍTICAS (P0 — bloqueia produção)

#	Problema	Onde	Risco	Como corrigir
C1	JWT_SECRET literal de dev em PROD e HML (idêntico)	.env:28-31, .env.hml:28-31	Qualquer um forja JWT ADMIN	Gerar 64+ bytes aleatórios distintos por ambiente, rotacionar, invalidar sessões
C2	SECRET_ENCRYPTION_KEY ausente → SMTP em texto puro no DB	.env falta, secret.service.ts:32-44	Senhas SMTP corporativas vazam em qualquer dump	Gerar chave (openssl rand -hex 32), migração que re-criptografa secrets existentes
C3	Senhas de DB, LDAP, Qive em .env em share de rede	.env, .env.hml	Vazamento via share/snapshot/ex-funcionário	Mover pra Vault/Credential Manager ou variável de ambiente do serviço
C4	Modo demo ativável em PROD a partir da tela de login	LoginPage.tsx:164-183, lib/api.ts:88	Atacante vira "ADMIN" mockado, confusão operacional total	Gating por import.meta.env.DEV ou VITE_ENABLE_DEMO, tree-shake em build prod
C5	refreshToken salvo em localStorage mesmo em modo cookie httpOnly	lib/auth.tsx:22,111,139,174,199	XSS exfiltra refresh → sessão indefinida	Backend mandar refresh por cookie httpOnly, frontend remover setItem(REFRESH_KEY)
C6	SQL injection latente — \$queryRawUnsafe com \${erpDbName} sem allow-list	linx-erp.service.ts (21 chamadas), legacy-orders.service.ts, erp-back-sync.service.ts, fiscal-documents.service.ts, product-orders-pa.service.ts	Cross-DB SQLi se ADMIN/DBA injetar valor malicioso em Company.erpDbName	Aplicar safeDbName() (já existe em financial.service.ts:85-91) em todos os lugares
C7	gravarPedidoCompra sem \$transaction — pedido parcial no Linx	linx-erp.service.ts:310-437	Cabeçalho gravado, itens faltando, recovery por OBS não cobre itens	Conversar com DBA Linx sobre XACT_ABORT ON + BEGIN TRAN controlado; ou strategy 2-fase

#	Problema	Onde	Risco	Como corrigir
C8	\$transaction(async() => this.prisma...) falso transacional	product-orders-pa.s ervice.ts:118-139, 198-217, 331-376	Aprovação/rejeição grava só metade — log inconsistente	Trocar this.prisma por tx recebido no callback
C9	DEMO_MODE bypassa autenticação backend sem checar senha	auth.service.ts:131-172, auth.controller.ts:375-404	Se DEMO_MODE_ENABLED=true em PROD por engano, qualquer um vira ADMIN	if (NODE_ENV==='production') throw ForbiddenException em loginDemo
C10	Deploy manual via Start-Process node sem PM2/auto-restart	RUNBOOK.md §2,§5	Crash do backend = downtime invisível até alguém reportar	Instalar PM2 + pm2 -windows-startup + pm2-logrotate
C11	Sem backup documentado de P2P_DB	runbook, schedule	Perda total em corrupção/disk failure/ransomware	SQL Server Agent Maintenance Plan + restore mensal de teste
C12	NODE_ENV=development no .env que é PROD	backend/.env:7	Stack traces vazam, optimizations off, Swagger pode ficar exposto	Trocar pra production, condicionar Swagger setup a non-prod

ALTAS (P1 — corrigir antes do go-live)

#	Problema	Onde
A1	rawXmlBase64 (NFe completa) sem controle por role + sem expurgo	schema.prisma:241, fiscal-documents.service.ts:1250-1261
A2	Audit log armazena response body com PII parcial e sem TTL	audit.interceptor.ts:24,73-110
A3	Cron qive-nfe-sync sem lock distribuído, estado em memória	fiscal-documents.service.ts:86-99, 360-524
A4	NFe parser por regex falha em XMLs com namespaces/CDATA	nfe-parser.ts:51-83
A5	JwtStrategy.validate() faz query Prisma por request HTTP	jwt.strategy.ts:43-63
A6	CORS com credentials:true aceita lista quebrada de origens	main.ts:43-55
A7	fund-requests.lastErpError pode vaziar SQL do Linx pra UI	linx-erp.service.ts:847, schema.prisma:927-928
A8	Account logout não conta tentativas em username inexistente; LDAP sem Turnstile	account-logout.service.ts:33-68, local-auth.service.ts:178-191
A9	findExistingSvByObs usa LIKE sem escapar %/_	linx-erp.service.ts:1086-1100
A10	Sem JwtAuthGuard global via APP_GUARD — risco de endpoint sem guard	app.module.ts:86-90
A11	Pages monolíticas 600-1055 LOC com lógica de negócio embutida	RequisitionFormPage.tsx, RequisitionDetailPage.tsx, TeamsPage.tsx, LoginPage.tsx, QuotationsCard.tsx, FiscalDocumentDetailPage.tsx, ContasPagarPage.tsx, PaOrderDetailPage.tsx
A12	Bug de rota — link de PC vinculado quebrado	FiscalDocumentDetailPage.tsx :221 (rota /pedidos-compra/ não existe)
A13	SupplierCombobox duplicado divergente	pages/financeiro/SupplierCombobox.tsx vs pages/requisitions/SupplierCombobox.tsx
A14	Search dispara query a cada keystroke (sem debounce)	RequisitionsListPage.tsx, ContasPagarPage.tsx, várias listas financeiras

#	Problema	Onde
A15	Jargão técnico vazando ("Lançamento", "ITP", "IAD", códigos crus)	ContasPagarPage.tsx:262-264, PaOrderDetailPage.tsx:54-63, sidebar Financeiro
A16	confirm() nativo em ações destrutivas/aprovação	AttachmentsSection, DelegationsPage, PositionsPage, TeamsPage, UsersPage, PaOrderDetailPage:304 (aprovação financeira!)
A17	loginStore e loginDemo forçam setEnvironment('PROD')	lib/auth.tsx:158,190
A18	Bundle único 791KB sem code-splitting + ReactQueryDevtools em PROD	App.tsx (50+ imports síncronos), main.tsx:4,16
A19	Sem timeouts em chamadas fetch à Qive	qive-client.service.ts:51,86,157,223,240
A20	Pool de conexão SQL Server com defaults (max:10)	prisma.service.ts:25-32
A21	CI roda lint com continue-on-error: true	.github/workflows/ci.yml:48
A22	Cobertura de teste praticamente nula	5 specs backend (1 stub), 0 frontend, 1 e2e
A23	Migrations PROD via Prisma vs HML via script paralelo — drift latente	apply-hml-migrations.js
A24	Sem rotação de logs (prod.log, hml.log, backend.log)	RUNBOOK.md:78-82
A25	Sem error tracking (Sentry/DataDog/NewRelic)	global
A26	Swagger pode estar exposto em PROD	main.ts:18-22
A27	@nestjs/bullmq + bullmq + ioredis instalados mas zero uso	package.json
A28	Tabela FiscalDocument.rawXmlBase64 vai inflar rápido (~10GB em 2 anos)	schema.prisma
A29	apply-erp-views.js CREATE OR ALTER VIEW em PROD sem snapshot	backend/apply-erp-views.js
A30	Throttler default em memória (não distribuído)	app.module.ts:51

MÉDIAS (P2 — até 60 dias após go-live)

- M1. SMTP fallback loga token de reset bruto em logs (`local-auth.service.ts:251-254`)
- M2. `audit.interceptor.ts` confia em `req.body.companyId` sem validar acesso
- M3. Schema com soft delete inconsistente — só 9 de 39 models têm `deletedAt`
- M4. `entityId String @db.UniqueIdentifier` no `audit_log` mas `interceptor` pega `id` que pode ser string não-UUID
- M5. `companies.cnpjRaizes` JSON em `NVarChar(500)`, parse ignora erro silencioso
- M6. `LdapAuthGuard` retorna `Record<string,unknown>` sem validar mail
- M7. Sem `retry/backoff` em chamadas Linx — uma falha mata o cron
- M8. `cookie-parser` sem signed cookies
- M9. Vários services com `as any` no Prisma `where/orderBy`
- M10. `validatePassword` aceita senhas fracas (`Aaaaaa1!`), sem `zxcvbn`
- M11. `bcrypt rounds = 10` (limiar inferior, recomendado 12)
- M12. `passwordSetupToken` SHA-256 puro (preferir HMAC-SHA256 com secret)
- M13. `LinxErpService.moedaPadraoCache` em memória — perde no restart
- M14. Estado `localStorage` espalhado em 9 chaves com convenção inconsistente
- M15. `queryClient.clear()` indiscriminado em todos os logins
- M16. Pouco uso de `useMemo/useCallback` (31 ocorrências em todo o projeto)
- M17. Acessibilidade fraca — 22 ocorrências total de `aria-*/role=`
- M18. `<TableRow onClick>` sem `tabIndex/role="button"` — não navegável por teclado
- M19. Erros silenciosos no logout (`auth.tsx:209-211` `swallow exception`)
- M20. `lib/demo/handlers` com 256+ `any` (aceitável em mocks, mas zero tipagem)
- M21. `Inputs type="number"` em campos monetários — perde formatação BR
- M22. Mensagens genéricas em 30+ toasts ("Falha", "Tente novamente")
- M23. Sem health check de dependências externas (AD, Linx, Qive)
- M24. `helmet` desabilita CSP/COEP em PROD pra Swagger funcionar
- M25. Vite build sem chunking explícito (`manualChunks`)
- M26. Vários `console.log` em `main.ts:69-70` — usar Logger do Nest
- M27. Pílulas de status às vezes via Badge, às vezes via `span inline` — inconsistente
- M28. Tooltip nativo `title=""` em vez do componente Tooltip do projeto

BAIXAS (P3 — backlog)

- B1. `lucide-react@^1.16.0` — verificar versão real e vulnerabilidades
- B2. Sem nenhum teste `.test.tsx` no frontend
- B3. `vite.log` no root (possivelmente comitado por engano)
- B4. Scripts ad-hoc `inspect-.ts/seed-.js` versionados misturados com prod
- B5. `bullmq/@nestjsjs/bullmq` declarados mas não usados (~20MB)

- B6. console.log em main.ts (usar Logger)
- B7. README desatualizado ("React 18" mas é 19, "PM2" mas usa Start-Process)
- B8. pad() em linx-erp.service.ts:103-105 só faz slice, nome enganoso
- B9. UUID v14 declarado vs crypto.randomUUID() usado — mistura
- B10. upsertParsed (linha 585) e upsertParsedV2 (linha 532) coexistem, só V2 em uso (código morto)
- B11. setTimeout() => navigate('/login'), 2000) em SetupPasswordPage.tsx:89 sem cleanup
- B12. ScheduleModule sem serverInstance único — PM2 cluster rodaria cron 2x

3. Classificação por severidade — totais

Severidade	Backend	Frontend	Infra/Integração	Total
CRÍTICA	6	3	3	12
ALTA	12	8	10	30
MÉDIA	22	13	10	45
BAIXA	10	10	8	28
TOTAL	50	34	31	115

4. Plano de correção por fases

Fase 0 — Bloqueio imediato (3-5 dias)

Endereça os 12 críticos. Tem que rolar antes de subir pra qualquer usuário não-piloto.

- Gerar JWT_SECRET, JWT_REFRESH_SECRET, SECRET_ENCRYPTION_KEY distintos por ambiente, mover pra fora de .env em share de rede.
- Rotacionar todas as credenciais que possivelmente vazaram: senha do integrador no SQL Server, conta P2P Service no AD, API key Qive.
- Gating do modo demo: gating por env no frontend (tree-shake em prod) E no backend (if NODE_ENV===production throw ForbiddenException).
- Frontend modo cookie: parar de gravar refresh em localStorage, backend mandar refresh por cookie httpOnly.
- Aplicar safeDbName() em todos os services com cross-DB.
- Trocar this.prisma por tx recebido nos \$transaction(async () => ...) em PA.
- PM2 com auto-restart + log rotation.

8. Backup diário do P2P_DB + teste de restore.
9. NODE_ENV=production no .env PROD + condicionar Swagger a non-prod.
10. Resolver gravarPedidoCompra parcial: 2-fase ou consenso com DBA Linx.

Fase 1 — Antes do go-live geral (2-4 semanas)

Endereça os 30 altos. Sem isso, o sistema sobe mas com risco operacional alto.

- Segurança/autorização: A1, A2, A6, A7, A8, A10
- Performance/escalabilidade: A3, A5, A18, A19, A20, A28, A30
- ERP/Linx robustez: A4, A9, A23, A29
- Frontend bugs e UX: A11 (extrair useRequisitionForm mínimo), A12 (bug rota), A13, A14, A15, A16, A17
- Build/CI: A21, A22 (cobertura mínima 30% nos services do Linx), A24
- Observabilidade: A25, A26, A27 (decidir: usa BullMQ ou remove)

Fase 2 — Estabilização (60-90 dias após go-live)

Endereça as 45 médias. Refinamento, hardening, qualidade.

- Schema: soft delete consistente, entityId tipagem (M3, M4)
- Frontend: extração de componentes (Pages → hooks + sections), acessibilidade (M14-M28)
- Backend: senha forte, bcrypt 12, signed cookies, retry em integrações (M7, M8, M10, M11)
- DevOps: health checks completos, manual chunks Vite, error tracking robusto

Fase 3 — Polish/backlog (contínuo)

Endereça as 28 baixas. Limpeza de código morto, scripts ad-hoc, docs, dependências não usadas.

5. Checklist do que precisa ser corrigido antes de produção

Segurança e secrets

- [] Gerar JWT_SECRET único de 64+ bytes pra PROD (diferente do HML)
- [] Gerar JWT_REFRESH_SECRET único de 64+ bytes pra PROD
- [] Gerar SECRET_ENCRYPTION_KEY (openssl rand -hex 32) e migrar secrets existentes
- [] Rotacionar senha do user integrador no SQL Server (PROD e HML)
- [] Rotacionar senha LDAP "P2P Service"
- [] Rotacionar QIVE_API_ID/QIVE_API_KEY
- [] Confirmar .env* não está no Git (git log -- .env) — se estiver, purgar histórico
- [] Adicionar .env.example sem valores reais
- [] Mover secrets de .env em share de rede pra variável de ambiente do serviço Windows ou Vault
- [] Gating do modo demo no frontend (import.meta.env.DEV ou VITE_ENABLE_DEMO)
- [] Gating do modo demo no backend (if NODE_ENV==='production' throw)
- [] Banner "MODO DEMO" persistente quando ativo (defesa em profundidade)
- [] Refresh token via cookie httpOnly (não localStorage) no modo cookie
- [] safeDbName() aplicado em linx-erp, legacy-orders, erp-back-sync, fiscal-documents, product-orders-pa
- [] JwtAuthGuard global via APP_GUARD com @Public() decorator

- [] RolesGuard global + auditoria endpoint a endpoint
- [] Restringir getXml/getDanfe por role
- [] Adicionar Turnstile no login LDAP
- [] Validar FRONTEND_URLS por regex no boot
- [] Swagger desabilitado em PROD
- [] NODE_ENV=production no .env PROD

Integração ERP

- [] Fix \$transaction falso em product-orders-pa.approve/reject/reschedule (usar tx)
- [] Validar com DBA Linx se XACT_ABORT ON + BEGIN TRAN resolve trigger conflict
- [] Mecanismo de reconciliação pra pedido parcial gravado (count itens vs po.items.length)
- [] Timeout (AbortSignal.timeout(30_000)) em todos os fetch Qive
- [] Retry com backoff em chamadas Linx
- [] Trocar parser regex de NFe por fast-xml-parser (ou aceitar limitações documentadas)
- [] Escapar %/_ em queries LIKE manual

Performance & escalabilidade

- [] Pool Prisma min:5, max:30, idleTimeoutMillis:30000
- [] Cache LRU 5min em JwtStrategy.validate() (1 query/req → 1 query/5min/user)
- [] Lock distribuído pro cron Qive (campo running no FiscalDocumentSyncState com watchdog 2h)
- [] Persistir syncStates em DB ou Redis (não memória)
- [] Throttler com storage Redis (já tem ioredis instalado)
- [] manualChunks em vite.config.ts
- [] React.lazy em rotas pesadas (admin/, financeiro/, fiscal/, pa/, legacy-orders/*)
- [] ReactQueryDevtools condicionado a import.meta.env.DEV
- [] Debounce 300ms em search bars

Deploy & operações

- [] PM2 instalado + pm2-windows-startup + pm2-logrotate
- [] Backup diário do P2P_DB (Maintenance Plan)
- [] Teste de restore mensal
- [] Sentry SDK no backend e frontend
- [] Health check Terminus cobrindo DB + AD + cross-DB Linx + Qive
- [] Documentar runbook de migração (Prisma deploy vs apply-hml-migrations.js)
- [] Resolver senha HML pra usar prisma migrate deploy direto (aposentar script paralelo)
- [] CI: remover continue-on-error do lint
- [] Pelo menos 30% cobertura de teste em LinxErpService

Frontend bugs e UX

- [] Bug rota: /pedidos-compra/ → /pedidos/ em FiscalDocumentDetailPage:221
- [] Remover setEnvironment('PROD') forçado em loginStore
- [] Substituir 5 confirm() nativos por ConfirmDialog (especialmente PaOrderDetailPage:304)
- [] Unificar SupplierCombobox em components/
- [] Mensagens de erro específicas em vez de "Erro inesperado. Tente novamente."

- [] Renomear labels em ContasPagar pra terminologia falante ("Nº do título" em vez de "Lançamento")

Dados & retenção

- [] Externalizar rawXmlBase64 pra storage (S3/MinIO/FS) ou GZIP
- [] Política de retenção audit_logs (cron expurgo > 1 ano)
- [] Truncar audit_logs.after a 16KB
- [] Estender regex de PII no AuditInterceptor (email, endereço, telefone, CEP)
- [] lastErpError passar por translateLinux() antes de salvar

6. Recomendações de arquitetura

1. **Criar LinxQueryService central.** Hoje 5 services duplicam o padrão [\${db}].dbo.X e a lista whitelist de erpDbs. Centralizar com método withErpDb(dbName, sql, params) que valida allow-list e parametriza placeholders. Reduz superfície de SQL injection e padroniza retry/timeout.
2. **Service layer para domínio fiscal.** FiscalDocumentsService cresceu pra >1k linhas com 4 responsabilidades (sync, fetch-on-demand, candidates, query). Quebrar em FiscalSyncService (cron + paginator), FiscalLookupService (candidates/legacy), FiscalQueryService (findAll/findOne/list).
3. **Padronizar identidade.** Hoje há AuthService, LocalAuthService, StoreAuthService com chamadas cruzadas e cookies inline divergentes. Definir interface IdentityProvider com login()/refresh()/logout()/me() e implementações LDAP/Local/Store/Demo. Cookies via helper único.
4. **Estado de cron em DB, não memória.** syncStates: Map, moedaPadraoCache: Map, companies.cnpjRaizes em JSON — tudo isso quebra em multi-instância. Mover pra DB ou Redis com TTL explícito.
5. **Constantes/enums TS em vez de strings mágicas.** 'COMPRAS_CONSUMIVEL', 'PENDING', 'A', 'E' aparecem inline. Criar const LINX_STATUS = { ATIVO: 'A', ENCERRADO: 'E', ... } as const.
6. **Schema lockfile + runtime strict.** Habilitar tsc --strict --noUncheckedIndexedAccess. Trocar Prisma where: any por Prisma.FiscalDocumentWhereInput.
7. **Pages-thin + sections + hooks.** Padronizar: pages/x/XPage.tsx (50-100 LOC, só compõe), pages/x/sections/ (cards/blocos visuais), pages/x/dialogs/ (modais isolados), pages/x/use-x.ts (custom hook com mutations + estado). RequisitionFormPage (1055 LOC) é o protótipo do que NÃO fazer.
8. **Migrar crons in-process pra BullMQ.** Já tem bullmq+@nestjs/bullmq+ioredis instalados sem uso. BullMQ resolve A3/A30 (lock distribuído natural). Crons "lite" via SO Task Scheduler chamando endpoint admin.
9. **Externalizar arquivos.** FiscalDocument.rawXmlBase64 no DB é antipattern. Attachment.storageKey já segue o padrão certo — replicar.
10. **CI rigoroso.** Lint bloqueante, testes obrigatórios, build com cache, deploy artifact assinado.

7. Recomendações de segurança

1. **Defesa em profundidade.** Mesmo com RolesGuard, parametrizar queries. Mesmo com cookie httpOnly, signed cookies. Mesmo com Throttler, Cloudflare na frente.
2. **Zero-trust em integrações.** Endpoint Qive com retry/timeout/circuit breaker. Linx queries com whitelist de DBs + parâmetros + log de execução.

3. **Rotação rotineira de secrets.** JWT_SECRET/SECRET_ENCRYPTION_KEY/SMTP/LDAP/Qive — calendário semestral.
 4. **Audit ≠ Log.** Audit log com PII redactada (estender regex), TTL definido (1 ano), índice por entity. Separar de logs operacionais (que vão pra Sentry/arquivo).
 5. **Rate limit por user + IP, não só IP.** Cobre credential stuffing distribuído.
 6. **Detecção em vez de reação.** Sentry + alertas pro Slack/email. Health check externo (UptimeRobot, etc.).
 7. **Inventário de superfície.** Documentar TODOS os endpoints públicos (Swagger gera, mas precisa estar atualizado e fora de PROD). Auditar guards.
 8. **LGPD.** Definir retenção legal de audit_logs, fiscal_documents.rawXmlBase64, dados de usuário. 5 anos para fiscal (regra contábil); 1 ano para audit pode ser suficiente.
 9. **Demo mode é vetor de ataque.** Em PROD, tem que ser impossível ativá-lo — não basta esconder o botão. Tree-shake total em build prod via flag.
 10. **Plano de resposta a incidente.** Quem é avisado se vazar? Como rotacionar tudo em <1h? Documentar.
-

8. Recomendações de UX/UI

1. **Glossário visual.** Tabela de "termos técnicos vs termos do usuário" pra cada perfil (Operador, Fiscal, Financeiro, Comprador). Aplicar nos labels. Ex.: "Lançamento" → "Nº do título"; "Posição" → "Conciliação"; "DDA" → "Boletos importados".
 2. **Substituir confirm() nativo.** ConfirmDialog já existe — 5 substituições diretas. Em aprovação de PA, dialog com motivo opcional + atalho de teclado.
 3. **Banner persistente em modo demo.** Vermelho, no topo, com texto "DADOS SIMULADOS — NÃO É PRODUÇÃO". Não pode ser cosmético; precisa estar evidente.
 4. **Loading skeletons.** Hoje a maioria das páginas mostra "Carregando..." em texto cinza pequeno. Skeleton (Radix Skeleton) padroniza e reduz CLS.
 5. **Empty states com call-to-action.** "Nenhum pedido encontrado" → adicionar "Criar requisição" ou "Limpar filtros".
 6. **Toast com variants completos.** Adicionar warning (amarelo) além de default/success/destructive. Em ações sem erro mas ambíguas (sync já rodando, etc.) cabe.
 7. **Debounce universal em searches.** 300ms commit, com indicador visual durante a digitação.
 8. **Mensagens de erro específicas.** Em vez de "Falha ao salvar", contextualizar: "Não foi possível salvar a requisição (rede instável). Tente novamente em alguns segundos."
 9. **Acessibilidade base.** aria-label em botões ícone-only. TableRow com role="button" + tabIndex. Foco programático em modais. Contraste auditado.
 10. **Code-splitting reduz primeiro paint.** Bundle 791KB → ideal <300KB pro shell + chunks por seção. Usuário de loja em 3G ruim agradece.
-

9. Recomendações de testes

Hoje a cobertura é praticamente zero. Não significa que precisa de 80% — significa que caminhos críticos precisam de teste.

Mínimo viável antes de PROD:

1. Unit tests no LinxErpService (criticidade máxima):

- gravarPedidoCompra com mock de \$queryRawUnsafe — verifica que falha entre cabeçalho e itens é detectada
- gravarSolicitacaoVerba (idem)
- criarFornecedorDeQuotation idempotência por CNPJ
- safeDbName (deve aprovar GUESS_PRODUCAO, rejeitar ' ; DROP TABLE X --)
- parseNfeBase64 com 5 fixtures reais (NFe 4.00 com e sem prefixo namespace, com/sem CDATA)

2. Integration tests dos endpoints fiscal-documents:

- /admin/sync dispara background + status reporta progresso
- /fetch-by-chave idempotente
- /:id/link-legacy valida companyId match

3. E2E Playwright estendendo o smoke:

- Login local + LDAP (com mock LDAP)
- Fluxo criar requisição → enviar → aprovar
- Sync manual da Qive e ver progresso no banner

4. Frontend unit tests (priorizar):

- extractApiMessage com 3 formatos do Nest
- auth.tsx flow de cookie vs bearer
- lib/demo/state setEnv/clearDemo
- Form de requisição: validação Zod + cálculo de total

5. Performance tests sob carga simulada:

- 50 users concorrentes em dashboard financeiro → pool DB satura?
- Cron Qive + 20 reqs paralelas → P95 OK?

Estratégia: começar pelo LinxErpService (cada bug ali = problema fiscal real). Depois fiscal-documents. Depois UI crítica. Frontend Playwright e2e é mais barato que React Testing Library a esta altura — investir nele.

Metas razoáveis:

- Antes go-live: cobertura $\geq 30\%$ em integration/ e auth/
- 60 dias após: $\geq 50\%$ global
- 6 meses: $\geq 70\%$

10. Perguntas que precisam ser respondidas antes de mexer no código

Sem essas respostas, várias correções viram chute. Em ordem de impacto:

1. **.env está versionado no Git?** `git log -- backend/.env backend/.env.hml` resolve. Se sim, todos os secrets já vazaram historicamente — rotação é obrigatória e urgente.
2. **A trigger LXI_COMPRAS do Linx aceita BEGIN TRAN externo com XACT_ABORT ON?** Precisa do DBA HRG3 pra validar. Sem isso, não consigo blindar `gravarPedidoCompra`.
3. **Qual o contrato Qive — por NFe consumida ou por chamada API?** O cron horário + manual + 2 ambientes podem dobrar gasto. Cota anual conhecida?
4. **Existe WAF (Cloudflare/Imperva) na frente do P2P?** Muda apetite a Turnstile/CORS/rate limit.
5. **PM2 vai rodar em cluster ou single instance?** Define se preciso resolver state-em-memória ou se posso adiar.
6. **Existe HTTPS + reverse proxy (IIS/Caddy/Nginx)?** Cert SSL onde? Logs do proxy contam pra observabilidade?
7. **Retenção legal de audit_logs e NFes (rawXmlBase64)?** Define política de expurgo.
8. **Quem responde pager noturno se o backend cair às 3h?** Define se PM2 auto-restart é suficiente ou precisa de pager.
9. **DEMO_MODE_ENABLED está como em PROD agora?** Verificar literal do .env produção.
10. **Permissions do user integrador no SQL Server.** É `db_datareader` em `GUESS_PRODUCAO` e `DB_HRG3`, ou tem mais? Define superfície de risco de SQLi cross-db.
11. **Quem é o público real de cada tela financeira?** Se forem 5 pessoas do financeiro, jargão Linx (ITP/IAD/DDA) é OK. Se forem operadores/gestores, refatorar.
12. **Recharts é usado em quantas telas?** Se só Dashboard, code-split dela isolada (recharts ~150KB).
13. **Plano de DR (Disaster Recovery)?** Servidor Windows morre → RTO/RPO?
14. **Modo demo: clientes finais ou só QA interno?** Define se vai pra build de PROD ou não.
15. **Tem teste de carga histórico?** Quantos users simultâneos esperados em pico?
16. **Há contratos/aprovações de que o P2P escreve em COMPRAS direto?** Caminho de rollback se pedido sair errado no Linx em PROD?
17. **canSwitchEnv faz proxy pra outro backend ou só dá acesso ao mesmo?** Se proxy, é vetor de confusão de ambiente.
18. **Quem é o owner de cada módulo?** Pra triagem de PRs e bugs.

Síntese final

Posso/devo fazer alguma coisa agora?

Você me pediu explicitamente pra não implementar nada e aguardar aprovação. Vou respeitar.

Sugestão de sequência de aprovação:

1. **Imediato (esta sessão se possível):** os 12 críticos. São quase todos infra/secrets/config — não exigem refactor pesado. Estimativa: 2-3 dias de trabalho focado.
2. **Próximo sprint (1-2 semanas):** Fase 1 (30 altos). Aí entra extração das Pages grandes, code-splitting, fix dos bugs UX, observabilidade básica.
3. **Próximos 2-3 sprints:** Fase 2 (45 médios). Polimento, refactor, testes.
4. **Backlog contínuo:** Fase 3 (28 baixos).

Recomendação prática: Comece pelos críticos C1, C2, C3 (secrets) HOJE — são quick wins de alto impacto. Depois C10, C11, C12 (deploy + backup + NODE_ENV). C4, C5, C9 (demo + refresh). C6 (SQLi). C7, C8 (transações ERP) precisam do DBA.

Aguardo seu OK pra começar — e qual subconjunto você quer atacar primeiro.

Documento gerado por Claude com análise de três agentes especializados (backend, frontend, infra/integrações) em paralelo sobre o working tree do projeto P2P em I:\p2p. Nenhum arquivo foi modificado durante a auditoria.